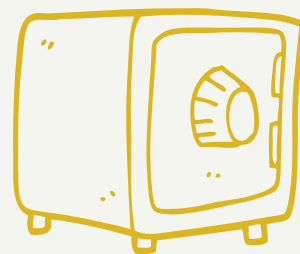


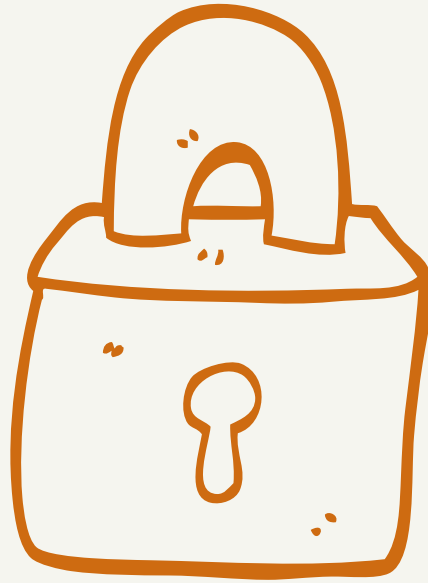
DEEP BREATH. IT'S TIME FOR AUTH.

AUTH



WEB DEVELOPER
BOOTCAMP

DEEP BREATH. IT'S TIME FOR AUTH.



AUTHENTICATION

WHAT IS IT?

Authentication is the process of verifying who a particular user is.

We typically authenticate with a username/password combo, but we can also use security questions, facial recognition, etc.



AUTHORIZATION

WHAT IS IT?

Authorization is verifying what a specific user has access to.

Generally, we authorize after a user has been authenticated.

"Now that we know who you are, here is what you are allowed to do and NOT allowed to do"

R U L E # 1
NEVER STORE PASSWORDS

DEEP BREATH. IT'S TIME FOR AUTH.

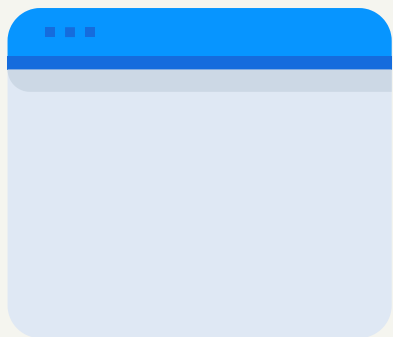
DEEP BREATH. IT'S TIME FOR AUTH.

R U L E # 1

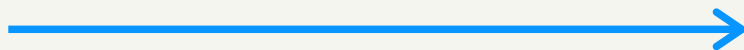
NEVER STORE PASSWORDS

```
{  
  username: 'kittycatluvr',  
  password: 'meowmeow999!',  
},  
{  
  username: 'geckoGuy',  
  password: 'lizard987'  
}
```

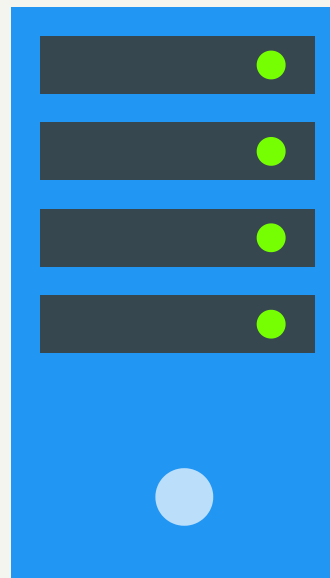
CLIENT



LOG ME IN WITH:
Username: 'geckoGuy'
Password: 'lizard987'



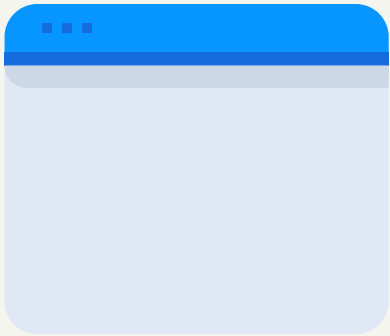
SERVER



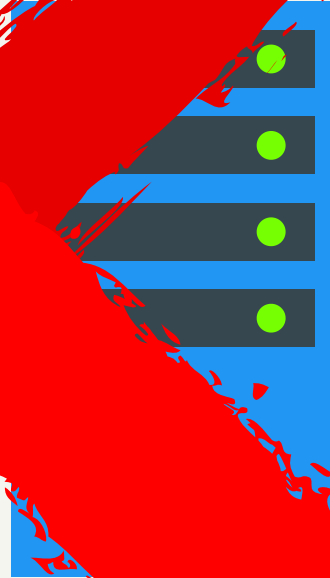
DATABASE

```
{  
  username: 'kittycatlvr',  
  password: 'meowmeow999!',  
},  
{  
  username: 'geckoGuy',  
  password: 'lizard987'  
}
```

CLIENT



LOG ME IN V
Username: 'geckoGuy'
Password: 'lizard987'



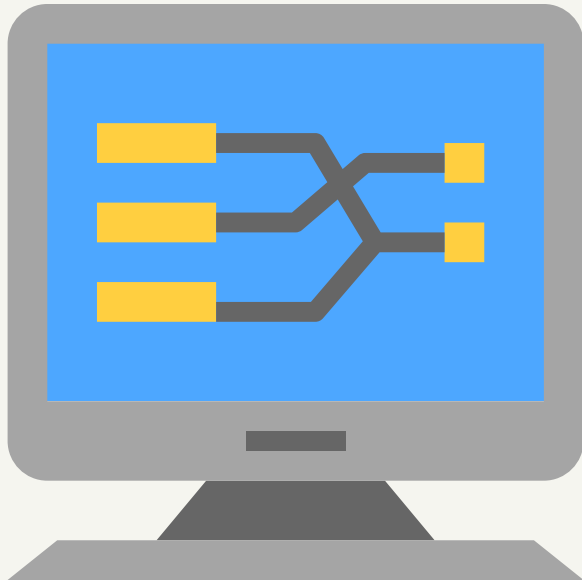
DATABASE

```
{  
  username: 'kittycatlvr',  
  password: 'meowmeow999!'  
},  
{  
  username: 'geckoGuy',  
  password: 'lizard987'  
}
```

No!







HASHING

THE SOLUTION!

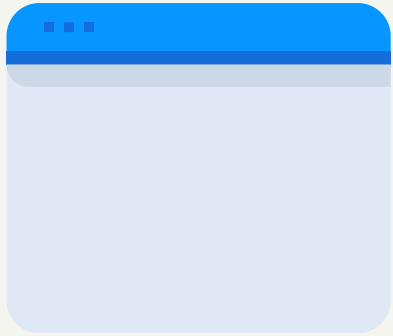
Rather than storing a password in the database, we run the password through a **hashing function** first and then store the result in the database.

HASHING FUNCTIONS

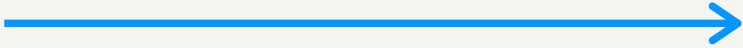
Hashing functions are functions that map input data of some arbitrary size to fixed-size output values.



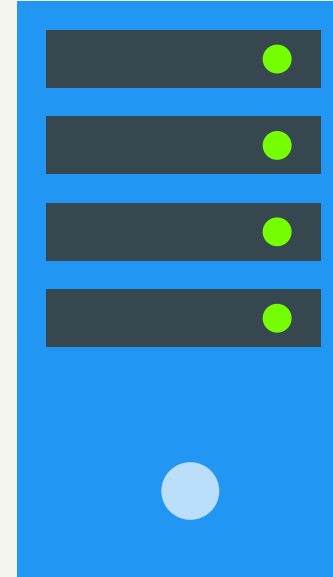
CLIENT



LOG ME IN WITH:
Username: 'geckoGuy'
Password: 'lizard987'



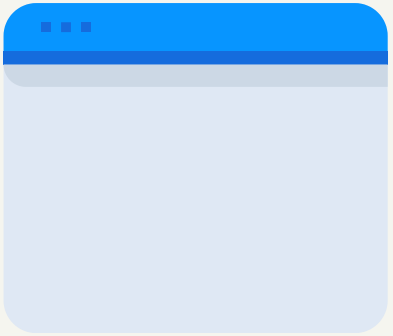
SERVER



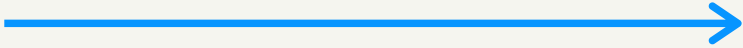
DATABASE

```
{  
  username: 'kittycatlvr',  
  password: 'd7offoab9a23ec5dba9075boe4de  
de8c2972ba933d6d5adf3a42abb6eod7a2da'  
},  
{  
  username: 'geckoGuy',  
  password: '07123eif482356c415f684407a3b87  
23e1ob2cbbcob8fcd6282c49d37c9ciabc'  
}
```

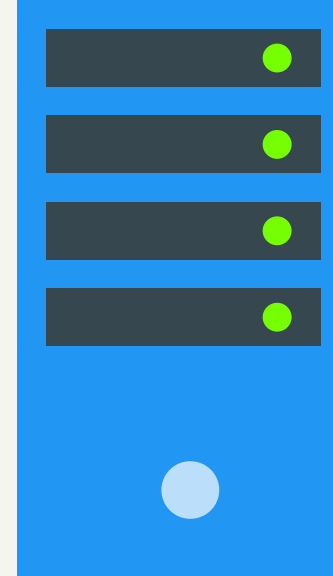
CLIENT



LOG ME IN WITH:
Username: 'geckoGuy'
Password: 'lizard987'



SERVER



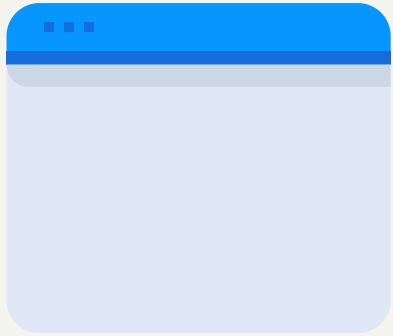
DATABASE

```
{  
  username: 'kittycatlvr',  
  password: 'd7offoab9a23ec5dba9075boe4de  
de8c2972ba933d6d5adf3a42abb6eod7a2da'  
},  
{  
  username: 'geckoGuy',  
  password: '07123e1f482356c415f684407a3b87  
23e1ob2cbbcob8fcd6282c49d37c9c1abc'  
}
```

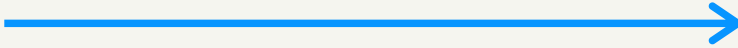
'LIZARD987'



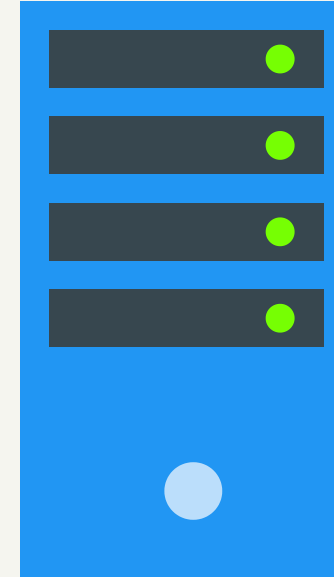
CLIENT



LOG ME IN WITH:
Username: 'geckoGuy'
Password: 'lizard987'



SERVER



DATABASE

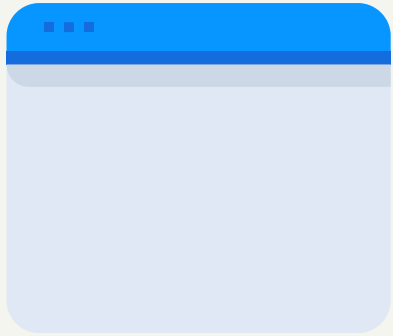
```
{  
  username: 'kittycatlvr',  
  password: 'd7offoab9a23ec5dba9075boe4de  
de8c2972ba933d6d5adf3a42abb6eod7a2da'  
},  
{  
  username: 'geckoGuy',  
  password: '07123e1f482356c415f684407a3b87  
23e10b2cbbcob8fcd6282c49d37c9c1abc'  
}
```

'LIZARD987'

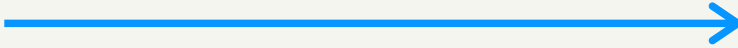


07123E1F482356C415F6844
07A3B8723E10B2CBBC0B8F
CD6282C49D37C9C1ABC

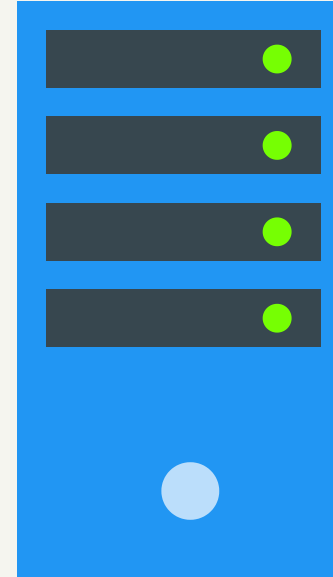
CLIENT



LOG ME IN WITH:
Username: 'geckoGuy'
Password: 'lizard987'



SERVER



DATABASE

```
{  
  username: 'kittycatlvr',  
  password: 'd7offoab9a23ec5dba9075boe4de  
de8c2972ba933d6d5adf3a42abb6eod7a2da'  
},  
{  
  username: 'geckoGuy',  
  password: '07123e1f482356c415f684407a3b87  
23e10b2cbbcob8fcd6282c49d37c9c1abc'  
}
```

IT'S A MATCH!

'LIZARD987'



FN



07123E1F482356C415F6844
07A3B8723E10B2CBBC0B8F
CD6282C49D37C9C1ABC

CRYPTOGRAPHIC HASH FUNCTIONS

1. One-way function which is infeasible to invert
2. Small change in input yields large change in the output
3. Deterministic - same input yields same output
4. Unlikely to find 2 outputs with same value
5. Password Hash Functions are deliberately SLOW

DEEP BREATH. IT'S TIME FOR AUTH.



S A L T S

AN EXTRA SAFEGUARD

DEEP BREATH. IT'S TIME FOR AUTH.



PASSWORD SALTS

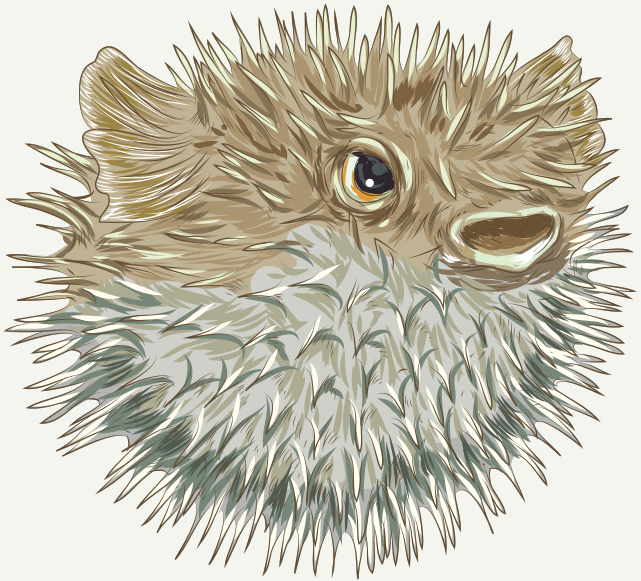
OMG THAT'S SO RANDOM!

A salt is a random value added to the password before we hash it.

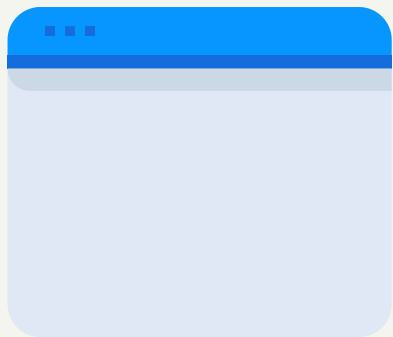
It helps ensure unique hashes and mitigate common attacks

B C R Y P T

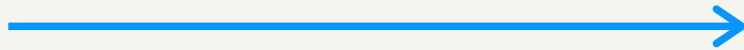
OUR HASH FUNCTION!



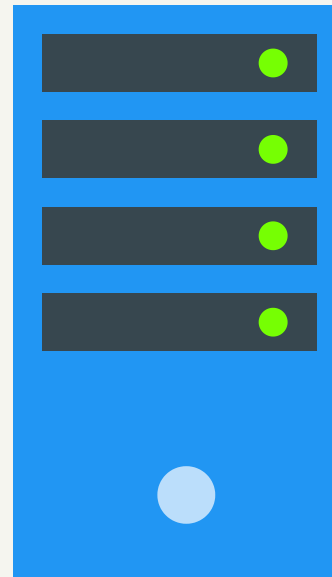
CLIENT



I have a cookie for you!
Session ID is 4



SERVER



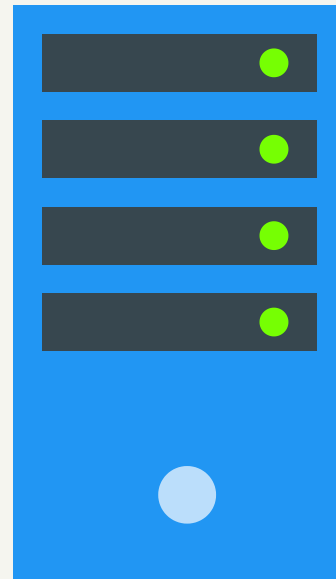
DATA STORE

```
{  
  id: 4,  
  shoppingCart: [  
    {item: 'carrot', qty:2},  
    {item: 'celery', qty:5},  
    {item: 'taser;', qty:99},  
  ]  
}
```

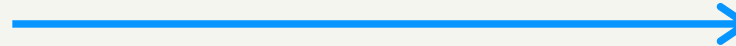
DATA STORE

```
{
  id: 3,
  shoppingCart: [
    {item: 'lime', qty:1},
    {item: 'la croix', qty:99},
    {item: 'lemon', qty:2},
  ]
},
{
  id: 4,
  shoppingCart: [
    {item: 'carrot', qty:2},
    {item: 'celery', qty:5},
    {item: 'taser;', qty:99},
  ]
},
{
  id: 5,
  shoppingCart: [
    {item: 'apple', qty:2},
    {item: 'onion', qty:5},
    {item: 'pear;', qty:9},
  ]
}
```

SERVER



Your session ID is 4



CLIENT

